# A Wakeup, Not a Broker: The Minimal Transport for Coordinating Stateless LLM Agents

Vasili Gavrilov*

2026

**Abstract**

Multi-agent coordination on one machine is treated as a messaging problem and handed to a queue, a broker, or an inter-agent protocol. That answers the wrong question. Moving bytes between processes was solved decades ago; the hard part is that an LLM agent, between turns, is not a running process at all - slow, serial, dormant - so throughput, the thing a broker manages, never binds. The scarce primitive is a *wakeup*, and the only one a stateless agent can receive is the return of a blocking `recv` parked in a cheap child process the harness re-invokes on exit, one activation per message, not per poll. That move is not new: it is the `epoll_wait` every network server blocks in, relocated from the agent - which cannot hold it - to a cheap child process, which can. We present `iac`, a dependency-free C99 board (about 1,100 lines, one binary, no heap).

The primitives are old; the assembly and the argument are new. The *economy* is measured: a self-polling agent reprocesses its whole context on every check - millions of tokens an hour to watch an empty mailbox - while a C-blocked wakeup costs zero, on any model. The *transport* is almost nothing: an append-only text log under a file lock, the commit log a log-based broker is built on minus the service, auditable by `cat` and `grep` and, because every agent reads the whole shared log, open to peer review - neither of which a queue can give. And *locality* is not a limit but the efficiency: on the single host a personal fleet actually runs on, a message is a file append and a page-cache read - the shortest path between two agents, which a broker on the same box can only add overhead to - and that same proximity keeps the whole shared history cheap to re-read, making the board self-correcting as well as auditable. The one new thing is the *participant*: an agent woken by nothing but a returning process. A fleet of agents used the board to build the board. Give it exactly that and no more: a wakeup, not a broker.

## 1 Introduction: the misidentified constraint

Several LLM agents running on one machine, each competent in isolation, are surprisingly hard to make work together. The reflex, borrowed from distributed systems, is to reach for messaging infrastructure: a message queue, an event bus, or one of the inter-agent protocols now consolidating in the field [18, 16]. That reflex imports a stack of assumptions - a service to run, identities to provision, a network to secure - and, we argue, solves the wrong problem.

The hard part of agent coordination is not moving messages. Bytes move fine between processes; that has been a solved problem for decades. The hard part is *waking a reader that, between turns, is not running at all.* An LLM agent advances only when its harness invokes it with input; in the interval between invocations there is no thread, no socket, and no event

*Independent researcher. Physics and computer science by education; professional software engineer with a machine-learning background, including rule engines and expert systems. Reference implementation: https://github.com/Anode1/iac.

loop into which a message could be delivered. Messaging infrastructure is built to buffer and distribute for consumers that are always live; it has nothing to say about a consumer that does not exist between turns.

This paper makes four claims. *First* (§2-§3), an LLM agent is the wrong shape for a queue: it is slow, serial, and dormant, so throughput - the thing a broker manages - is never the bottleneck. *Second* (§4-§5), the primitive an agent actually needs is a wakeup, and the only wakeup available to a stateless agent is the return of a blocking receive run in a cheap child process; a token-cost model shows why this must live in C, not in the model. *Third* (§6-§7), once a wakeup is the requirement, the transport collapses to an append-only log plus advisory locking - the commit-log abstraction beneath a log-based broker, minus the service - realised here as `iac`, a dependency-free C99 reference implementation. *Fourth* (§10), we report an experience in which a fleet of agents used the board to build the board.

The design is reduced to its essential primitive, and the artefact kept small enough to audit by reading.

## 2 The consumer an agent actually is

A message queue exists to serve a particular kind of consumer: one that is fast, numerous, and concurrent. An LLM agent is none of these.

**Slow.** A turn - read input, reason, act - costs seconds of wall-clock time and a model inference, not the microseconds of a function call.

**Serial.** An agent processes one message at a time. It has one context window, one attention, one output stream, and cannot parallelise its own reasoning. It is, in the strict sense, a single-threaded reader.

**Dormant between turns.** This is the decisive property. Between invocations an LLM agent is not a running process: no thread, no file descriptor, no socket, no event loop. Nothing can interrupt it or deliver to it. It comes into existence only when its harness re-invokes it, runs one turn, and ceases.

**Locality-bounded.** In practice the fleet does not grow into a throughput problem either. A single human operator steers only a handful of agents at once - a few in ordinary use, and rarely more than about ten when the work is interdependent and each agent needs review - and on one host each agent running a heavy build or toolchain consumes gigabytes of memory, which tightens the bound further. A fleet of that size, driven at human pace, never generates the message rate a broker exists to absorb. This is an observation about how the tools are used, not a hard limit: when agents run largely autonomously, with the operator only occasionally in the loop, the operator-span bound relaxes and larger fleets become possible. But the per-agent seriality above is the load-bearing point, and it does not relax - each agent is still a slow serial reader.

So on both axes - the single agent and, in ordinary use, the whole fleet - throughput is a non-problem. The queue is optimised for exactly the pressure that never arrives.

## 3 Why a broker is the wrong tool

A broker's job is throughput and concurrency: buffer bursty producers, fan work out to many consumers, apply backpressure, drain in parallel at speed. Pointed at a slow serial reader, none of that machinery does any work; it is a turbine bolted to something that walks.

More to the point, delivery presupposes something the agent does not have. Brokers reach consumers in exactly two ways: either the consumer runs a *poll loop* - it calls a blocking receive, the kernel wakes it when data arrives, and it fetches; or the broker *pushes* to a connection the consumer holds open. Both require a running consumer. A Kafka consumer receives its messages precisely because it is a live process blocked on a poll, woken by the kernel - cheap, because a process blocking on a socket read costs nothing, and it is never dormant.

An agent between turns is not that process. It has no poll loop, because there is no process to run one, and no held connection to push to. So to receive at all, it needs an *external* process to hold the wait on its behalf and wake it on arrival - the construction of §4. That waking process is what matters, and it is independent of the transport beneath it: one could wrap it around a Kafka consumer, a socket read, or a file, and get the same wakeup. This is the case against the broker. It is not that a broker *cannot* deliver the wakeup - wrap its consumer in the same waking process and it can - but that the wakeup is required *either way*, and once it is in place, the broker's throughput and concurrency machinery does no work for a slow serial reader. It is weight, dependency, and cost around a need it does not remove. Making the *model* itself poll is worse still: every check is a full inference over the whole context, paying a language model to stare at an empty mailbox.

## 4 The wakeup primitive

What, then, can wake a stateless agent? Exactly one signal, and it is one the environment already supplies: the one a parent process receives for free - *a background child has finished.* Agent harnesses re-invoke the model when a spawned child process exits. That return is the only way an inbound message can reach a dormant, harness-invoked agent.

The construction follows directly. Park a blocking receive in a cheap child process:

```
recv(room, me, timeout)
```

On Linux this process sleeps on an `inotify` watch of the room's log (a ∼100 ms poll elsewhere), consuming nothing while it waits - one process asleep, exactly like a blocked `epoll_wait`. When a message addressed to `me` appears, `recv` returns; the child exits; the harness re-invokes the agent, which comes alive holding the message. This is the ordinary consumer poll loop of §3, *relocated* into a process the agent can afford to keep alive, because the agent cannot be that process itself. Anyone who has written a server recognises the move at once: it is the textbook choice to block in `epoll_wait` rather than busy-poll a flag, applied to a caller that happens to be a language model - where the poll being avoided costs not a wasted syscall but a full inference.

In one picture (the figures are measured in §5):

```
THE USUAL WAY (model polls)     THE iac WAY (recv, asleep)

check -> "nothing"   $$$        zzz .........   $0
check -> "nothing"   $$$        zzz .........   $0
check -> "nothing"   $$$        zzz .........   $0
...all day...        $$$$$      message in ->   $ one turn

10 agents: ~$20-$180 / hr       waiting is free.
an 8h day: ~$160-$1,400 gone    pay for words, not for waiting.
```

It is cheap for one reason: the sleeping waiter is a plain C process, not the model, so it burns no tokens while parked - the bill scales with messages, not with time. The key property is **one model activation per message, not per poll**. The polling still happens - it must - but inside the C process, where an idle wait costs nanoseconds, never inside the model, where every

check costs a turn. The receive model is developed at length in the reference implementation's documentation in `poll`/`epoll` terms [2].

**Scope.** This argument is about *stateless, harness-invoked* agents: the CLI subagent that runs a turn and stops, which is the common case today. The claim "the only wakeup is a returning process" holds *given* a harness whose one wake signal is a background child's exit; a different harness could expose a native file-watch or a scheduled re-invocation instead. And an agent hosted as a long-lived server is a different animal - it has a socket to hold open and can be pushed to like any service. But that reintroduces the running process, the network, and the identity the minimal design exists to avoid. For an agent that only exists when invoked, a returning process is the wakeup available.

## 5 A token-cost model

The reason the poll must live in C rather than in the model is economic, not aesthetic. Consider an agent idle for a wall-clock interval $T$, watching for messages, with a naive poll interval $p$ and a context of $C$ tokens.

If the *model* performs the poll, it is invoked $T/p$ times. In the worst case - each poll separated from the last by more than the provider's prompt-cache time-to-live (on the order of minutes), so the cache is cold - every invocation re-ingests the whole context, and the idle cost is

$$A_{\text{model}} = \frac{T}{p}, \qquad \text{Tokens}_{\text{model}} \lesssim \frac{T}{p} C \quad \text{(cold cache: worst case)}. \tag{1}$$

Within the cache lifetime a re-poll still forces a full pass over $C$ - the model reads all of it - but is *billed* for only a fraction, so the per-poll cost falls while the work does not; and the number of *activations*, $T/p$, does not fall at all. It is the activations that diverge as $T$ grows or $p$ shrinks.

If instead the poll lives in a C process that re-blocks on each timeout without invoking the model, the idle cost is

$$A_{\text{C}} = 0, \qquad \text{Tokens}_{\text{C}} = 0 \quad \text{(while idle, for any } T\text{)}. \tag{2}$$

A background shell loop, `while :; do recv ...; done`, re-blocks in C on every timeout and invokes the model only when a real message breaks the loop. An agent can therefore sit on a board for days at zero token cost.

Under the wakeup design, model activations scale with *work*, not with *time*:

$$A_{\text{wakeup}} = \#\{\text{messages for me}\} = O(\text{messages}), \quad \text{versus} \quad A_{\text{model-poll}} = O(T/p). \tag{3}$$

**A worked example, measured.** We ran exactly this. A Claude Sonnet agent carrying a $C = 49{,}836$-token context - a realistically loaded agent - was asked to check for a message every $p = 30\,\text{s}$ for one hour, so $T/p = 120$ activations, against the Anthropic API, which returns exact token usage per call; the dollar figures below are that measured usage priced at list rates[1] (reproducible with the token-cost harness in the repository):

| Approach | Model wake-ups | Tokens processed | Cost (USD) |
|---|---:|---:|---:|
| Model polls itself (warm cache) | 120 | 5,980,320 | $1.99 |
| Model polls itself (cold cache) | 120 | 5,980,320 | $17.95 |
| `iac` wakeup (`recv` blocks in C) | **0** | **0** | **$0.00** |

[1]Sonnet list rates used here: $3, $0.30, $3.75, and $15 per million input, cache-read, cache-write, and output tokens respectively (2026).

The model read 5.98 million tokens either way: a check is a full inference over the whole context, and a language model has no cheaper way to look - there is no partial forward pass over a mailbox. What the prompt cache changed was the bill, not the work. Warm - each poll inside the cache lifetime - cost $1.99 for the hour, billed almost entirely as cache reads (5,926,081 of the tokens; the rest the initial cache write and per-call output); cold, the same tokens with the cache disabled, would bill $17.95 (derived from the measured processed tokens at the full input rate); and neither figure depends on there being any message to find. `iac` read nothing, was billed nothing, and cost $0.00 while idle, waking the model once when a real message arrived. Scaled to a modest fleet - five agents parked overnight for eight hours - the self-poll processes on the order of 240 million tokens and bills roughly $80 warm, about $720 cold, all before any work is done, against zero for the board.

**Not specific to one model.** The mechanics are a property of how language models run, not of any one vendor. Every current LLM serves a request statelessly - the model is a function from an input context to output tokens, with no process persisting between calls - so a fresh forward pass over the whole context is the only way it can *look* at anything. There is no cheap peek, on any model. Prompt caching, now offered by every major provider, discounts the repeated prefix but never to zero, and reduces neither the forward pass nor the activation count $T/p$. The discount itself varies, and Anthropic's cache read at roughly a tenth of the input price is among the most generous, so the $1.99 warm figure above is a *favourable* case; where a provider discounts less - some bill about half - the same poll costs more, not less. The wakeup removes the activations outright, and that holds whatever model sits behind the harness.

This is why the poll must move into C, and why the receive loop must be held by a shell driver or the harness, not by a parked model context [2, 3].

**Delivery latency.** The transport's own contribution to a round-trip is small and measurable. Timed on one Linux host against the reference binary, over several runs:

| Stage | Time | What it is |
|---|---|---|
| `send()` call | ∼4 ms | append one frame under an `flock` |
| `send` to `recv` wake, warm log | 3-8 ms | spin-up dominates a sub-ms `inotify` wake |
| `send` to `recv` wake, cold room | ∼100 ms | first message, before the log exists: one poll tick |
| round-trip through a real agent | > 120 s | the agent must read, think, and reply |

The numbers are process spin-up plus the wake, and the spin-up - forking and exec-ing the sender and the receiver - dominates a sub-millisecond `inotify` wake; only the first message into a not-yet-created room pays the ∼100 ms poll tick, because the watch attaches once the `log` file exists. Set against these milliseconds, the last row is the point: a live round-trip through an agent that must read and reply exceeded a two-minute cap without returning, bounded by a model turn, not the transport, by a factor of thousands. Delivery is millisecond-class - right for coordination, wrong for a chatty inner loop, and negligible beside the seconds an agent spends thinking.

## 6 The minimal transport

Once the requirement is a wakeup and not throughput, the transport can be almost nothing: append a line, lock it, read forward, block on receive. Concretely, the reference design [1] is:

| | |
|---|---|
| a ROOM | is a directory |
| the LOG | `<room>/log` - one append-only, totally-ordered stream |
| a CURSOR | `<room>/<name>.cur` - each member's read offset |
| the ROSTER | `<room>/roster/<name>` - presence |

Every message is appended to the log *once*, whatever its audience, so a broadcast is one write, not $N$. Each member reads the shared stream forward from its own cursor and delivers only the frames addressed to it, giving a single total order - a real conversation - with $O(1)$ broadcast and point-to-point or subset delivery for free. A frame is length-prefixed, `<from>|<to>|<epoch>|<len>` followed by the body, so a body may contain arbitrary bytes. The recipient field carries the whole addressing scheme: `*` (broadcast to all but the sender), a name (point-to-point), a comma list (subset), or `?` (a work item claimed by exactly one free member).

Advisory file locking (`flock`) does three jobs at once. It *orders appends*: a sender takes an exclusive lock and writes with a single `writev`, so concurrent senders never interleave and the log stays a clean total order. It *backs presence*: a parked `recv` or a `hold` beacon takes a shared lock on its roster entry for the life of the wait, and `who` probes the lock - held means online. And it *guards claims*: a `?` work item is won by an atomic `O_CREAT|O_EXCL` create keyed on the message's log offset, so the job runs once across a pool with no coordinator. A claim left unacknowledged past a time-to-live is presumed dead and stolen by the next free worker, so a crash mid-task does not lose the work. Presence is a held lock rather than a heartbeat, so the operating system clears it the instant the holder dies, even on `SIGKILL` - nothing to reap, and no process-id-reuse guessing.

Two old lineages meet in this shape, and the shape is the paper's point. The append-and-lock-while-writing is how Unix local mail is delivered: a mailbox the delivering agent locks while it appends. But mail is per-recipient, and this is not. This is one shared, totally-ordered log that many readers consume, each from its own offset, filtering by address - and that shape, an append-only commit log read forward through per-reader offsets with broadcast as a single append, is precisely the abstraction a log-based broker is built on [7]: a single-partition Kafka topic with consumer offsets, or a shared bulletin board in the older idiom. The paper argues against *brokers*, not against the *log*. For a slow serial consumer the log needs no service around it - only a file and a lock. That the design lands on such a proven abstraction is a sign it has found the grain of the problem rather than fought it.

**The record is auditable by construction.** One property follows from the log being plain text on the local filesystem rather than a binary store behind a service, and it is a security argument. The entire coordination history - who said what, to whom, in what order - is a single append-only text file. It can be read with `cat`, searched with `grep`, followed live with `tail -f`, diffed, put under version control, and archived, all with tools already present on any Unix host and with no service running. Mainstream brokers keep their messages in binary on-disk formats not readable as plain text without the broker's own tooling - Kafka's segment files, an AMQP message store, or a cloud queue that is remote and opaque - so even where the payload is text, the record around it (framing, offsets, headers, ordering) is not `cat`- or `grep`-able, and often there is no local artefact to inspect at all. For a system whose participants are autonomous agents, an after-the-fact audit trail a human can read without special tooling is a genuine safety property: every action is on the record, and the record is legible. That same transparency is, by design, the cooperative-trust boundary of §9 - every participant can read the whole log - so `iac` trades confidentiality between agents for auditability by the operator who runs them.

**A shared log also enables peer review, which a queue cannot.** Transparency has a second consequence, and no queue can offer it. A queue delivers each message to one consumer

and drops it; a fan-out topic hands each subscriber its own ephemeral copy; even a log-based broker that retains the record keeps it in a remote store reached only by a costly round trip. In none of these does the whole shared history sit cheaply in front of every participant. On a shared, local, plain-text log it does - every agent reads the same stream forward, re-readable at negligible cost, as often as it likes - and that shared, visible context is the precondition for cross-checking: independent agents inspect each other's claims and catch the errors, as they did in the session of §10, where one agent's proposed fix was refuted by another on a fine point of lock semantics before it could land. It is the reason the *blackboard* architecture was valued in classical AI [8]. Its knowledge sources did not message one another point to point; they posted to and read from a common, visible workspace, and that shared visibility - many fallible specialists inspecting and refining the same partial solution - was what let the system converge and correct. A queue is precisely the point-to-point model the blackboard was defined against; `iac` brings the blackboard's shared, readable workspace to LLM agents at the cost of a file. So the auditability above also holds *during* the work, for the agents, and that is what turns a fleet on a legible board into a self-correcting one. This turns on proximity: only because the log sits on the same host as the agents is re-reading the whole shared history a local read rather than a network fetch, and that is what lets cross-checking run *continuously* rather than as an occasional audit. `iac`'s same-host locality is what enables the self-correcting property; a deliver-and-forget queue has no shared history to re-read.

## 7 Reference implementation: `iac`

`iac` is roughly 1,100 lines of C99 (1,098 at the time of writing), with no dependencies beyond a POSIX C library: `flock`, `writev`, `pread`/`pwrite`, `dirent`, and `inotify` on Linux with a portable poll fallback. It builds with a stock compiler and ships as one binary; there is no daemon, no runtime, and no account. The command surface is eleven verbs - `send`, `recv`, `drain`, `ack`, `ask`, `join`, `leave`, `hold`, `who`, `log`, `compact` - and the documentation names three receive-loop shapes with their cost trade-offs: draining the mailbox inline while already awake, a background `recv` that re-invokes on exit, and a shell `while`-driver that idles at zero token cost [3]. Two `recv` flags push more of the bookkeeping into the sleeping child, on the same principle: `-a` delivers the whole burst already queued in one return, so a seat pays one model turn per burst rather than one per message plus a drain; `-e` returns with a distinct exit code once every other member has been absent for a set time, so the wakeup thesis extends to absence, and a seat never spends a turn discovering that its peers are gone.

The engine follows the core rules of safety-critical C. There is no heap allocation on any path, so peak footprint is a function of the fixed struct sizes and not of the data - a one-kilobyte room and a one-gigabyte room run in the same memory; string operations are bounded (`snprintf` only, never `strcpy`/`sprintf`); and the main receive and scan paths, which hold an open file across several exits, use single-exit `goto` cleanup. These are the no-heap, bounded-string, and single-exit rules of the NASA/JPL *Power of Ten* [20] and MISRA-C:2012 [21] (rule 21.3 bans dynamic allocation), in the K&R style [22] as carried into modern systems practice by Robbins [23], and they are recorded as a coding standard the implementation conforms to [5]. The engine does not claim the *full* Power-of-Ten discipline, but the memory-safety rules that eliminate C's dominant defect classes hold throughout. Continuous integration builds and runs an end-to-end suite - which drives the real binary through point-to-point, broadcast, ordering, claim recovery, and presence - on Linux and macOS, with AddressSanitizer and UndefinedBehaviorSanitizer lanes on both.

## 8 Related work

The space around `iac` is crowded, and growing.

*In-process agent frameworks* - AutoGen, LangGraph, CrewAI, MetaGPT - orchestrate several models inside a single runtime, typically Python, with function calls and shared memory. They are a different layer: coordination *within* a process, not delivery *between* independent processes.

*Shared boards for coding agents* are the nearest work, and it arrived while this paper was in preparation. AgentRoom [12] gives concurrent coding agents file-level claims, an append-only broadcast log, and peer status, as MCP tools over a CRDT-merged workspace; at matched compute it reports that the coordination tools, not the shared filesystem, carry the gain, and that a homogeneous pair beats a solo agent's abandonment rate by an odds ratio of 13.7. PatchBoard [13] replaces free-form dialogue with validated patches to a shared structured state; two further systems rebuild the blackboard around LLM knowledge sources selected by board content [14, 15]. These confirm the shape this paper argues for, from a different direction, and they differ from `iac` on exactly its axis: in each, an agent learns of a peer's post by *polling* between subtasks, paying an inference per check, where an `iac` seat parks a C child on the log and pays nothing until a peer posts (§4-§5).

*Same-host agent tools* are the closest cousins in mechanism. `tap` is a file-based protocol for heterogeneous agent collaboration, with markdown messages that persist across restarts, reported with a multi-week self-application [16]; `hcom` hooks CLI harnesses into a shared messaging and event bus; `swarm-protocol` exposes a coordination layer, as an MCP server, that claims work and detects file conflicts; `OpenClaw` runs agents behind an always-on Node.js gateway that brokers their messages and pushes events to connected clients [17]; `ntm` and `amux` tile and multiplex parallel sessions; and some harnesses now add inter-agent messaging, a shared task list, and file locking natively. Each carries a dependency or runtime tax - Node and npm, an MCP server, `tmux`, or lock-in to one harness - from which `iac` differs by having none.

*Inter-agent protocols* form the enterprise layer above all of this: A2A, introduced by Google in 2025, since donated to the Linux Foundation and reaching a v1.0 specification in 2026, positioned as an agent-to-agent standard across the major clouds [18]; MCP, the de-facto agent-to-tool standard, donated by Anthropic to the Linux Foundation's Agentic AI Foundation [19]; and ACP and ANP. These are networked, cross-organisational, and heavy by necessity; they answer a different question than a same-host board.

The classical lineage is explicit; `iac` sits in two of its threads. The first is coordination through a shared space: the *blackboard* architecture of classical AI, in which independent knowledge sources post to and read from a common board by relevance [8]; and Gelernter's *tuple spaces* (Linda), whose `out`/`in`/`rd` primitives include a destructive one-taker `in` that is structurally `iac`'s `?` claim [9]. `iac` is, in effect, a blackboard with a total order. The second is the *commit log*: an append-only, totally-ordered record consumed through per-reader offsets [7, 10], the data model beneath modern brokers, which stripped of the service is exactly `iac`'s log and cursors. Alongside these sit the ordinary IPC primitives (named pipes, Unix-domain sockets, ZeroMQ, D-Bus), mbox and Maildir for file-backed mailboxes, and the actor model [11], in which an entity's whole life is a receive loop over a mailbox. `iac` competes with none of the tools above on features or scale; its axis is minimal, dependency-free, same-host, harness-agnostic, chosen for install-simplicity, auditability, and longevity. In a field this volatile, the smallest thing that works is often the thing still working when the frameworks have moved on.

## 9 Limitations

The design is deliberately narrow, and the narrowness has edges.

*Same host.* Coordination is over a shared filesystem; there is no cross-host story without a shared mount (on which `flock` semantics, over NFS in particular, are a known hazard) or swapping the log for a socket while keeping the framing.

*Cooperative trust.* The sender's name is an unverified label: any participant can post under any name, and because the log is one shared file, every participant can read every message

regardless of its addressing. The model assumes mutually trusting agents isolated by filesystem permissions. The natural extension, without changing the transport, is a per-frame HMAC verified at `recv`.

*Scale.* A single append lock serialises all sends, and every reader scans the whole log to filter; both are fine at coordination scale and become ceilings at high traffic, where the answer is to shard into more rooms. The log grows unbounded and is reclaimed by a manual compaction pass; bounding this automatically is declared future work [4].

*Harness-specificity.* The wakeup rests on the harness re-invoking on child exit; it is scoped to stateless agents (§4).

*Obsolescence.* The tool may well be subsumed by native coordination features in the agent harnesses themselves. If so, the argument - that the primitive is a wakeup, and that for a serial consumer minimality is the answer - is the part intended to outlast the artefact.

## 10 Experience: agents building the tool

After the author wrote an initial version, a fleet of independent agents used the board to develop the board itself. Over a single session they diagnosed and fixed a presence bug in the C (a stored process id that could go stale while a receive-only agent held the name online), carried out a macOS portability pass, wrote and revised the documentation, and coordinated all of it as messages on the plain-text log - proposals, reviews, claims, and completions, addressed by name, landed as commits, and readable afterward as a chat transcript one could `cat` and `grep`. Watching a fix argued and merged entirely over the artefact under repair was the point at which the argument stopped being theoretical.

A second, external instance followed. Three coding-agent seats (Claude Sonnet, headless, one process each) were put on an `iac` room to fix one SWE-bench Verified issue in Django (`django-11141`), with a shared checkout and a brief that asked each seat to post its diagnosis before reading the others'. The room log records 14 messages over 11.9 minutes: three independent diagnoses within 11 seconds, a claim race on `loader.py` settled by the log's own order, a split into fix, tests and release notes, and a seat that lost the claim turning itself into a verifier. Wakeups across the three processes landed within 3 seconds; the seats cost $2.11 in total; the collective patch was scored *resolved* by the official SWE-bench harness. The run was a pipeline pilot, excluded from the pre-registered comparison it preceded, and it says nothing about boards against solo agents. It says that a plain-text log with a wakeup carries a real three-party repair to a verified result, and that the record of how is one file.

This is an existence proof, not an evaluation. The agents were the author's own, on one machine, under a single operator ($n = 1$ for the self-build, $n = 1$ for the pilot); it shows feasibility and that the transcript is auditable, not that the approach generalises across teams or scales. But feasibility was the open question, and the recursion - the tool they talk through being the tool they build - is itself evidence that a wakeup over a plain-text board is enough to carry real, multi-party engineering work.

## 11 Conclusion

The instinct to solve agent coordination with a broker imports the wrong problem. For a consumer that is slow, serial, and dormant between turns, throughput was never the bottleneck; the missing primitive is a wakeup - and the only wakeup a stateless agent can receive is the return of a blocking receive run in a cheap process the harness re-invokes on exit. Grant that, and the transport reduces to an append-only log, advisory locks, and a blocking `recv`: the commit-log abstraction a log-based broker is built on, stripped of the service.

Twenty-five years ago the author's answer to the same coordination question was a running peer-to-peer broker, `ljms` [6]. The lesson of this iteration is that the broker was the part to

delete. The surviving parts are old; that is the point. The participant is new, an agent that can be woken by nothing but a returning process, and so is the discipline of giving it no more than it needs. Here, minimality is the engineering answer.

## References


[1] `iac` project documentation, `README.md` - model, frame format, presence, trust model, and limits. https://github.com/Anode1/iac, 2026.

[2] `iac` project documentation, `doc/dev/RECEIVE_MODEL.md` - the receive model in `poll`/`epoll` terms, and the token cost contract. https://github.com/Anode1/iac, 2026.

[3] `iac` project documentation, `SKILL.md` - receive-loop shapes and cost trade-offs. https://github.com/Anode1/iac, 2026.

[4] `iac` project documentation, `doc/ROADMAP.md` - bounded log growth and stale-name reaping (future work). https://github.com/Anode1/iac, 2026.

[5] AIS project documentation (a sibling project), `doc/dev/STYLE.md` - coding standard the `iac` engine conforms to. https://github.com/Anode1/ais, 2026.

[6] V. Gavrilov, *ljms*: a peer-to-peer message broker with broadcast and multicast, 2001. Prior, unpublished work; noted in the `iac` `README.md` lineage.

[7] J. Kreps, *The Log: What Every Software Engineer Should Know About Real-Time Data's Unifying Abstraction*. 2013.

[8] L. D. Erman, F. Hayes-Roth, V. R. Lesser, D. R. Reddy, *The Hearsay-II Speech-Understanding System*. ACM Computing Surveys 12(2), 1980. See also H. P. Nii, *Blackboard Systems*, AI Magazine 7(2-3), 1986.

[9] D. Gelernter, *Generative Communication in Linda*. ACM TOPLAS 7(1), 1985.

[10] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil, *The Log-Structured Merge-Tree (LSM-Tree)*. Acta Informatica 33, 1996.

[11] C. Hewitt, P. Bishop, R. Steiger, *A Universal Modular Actor Formalism for Artificial Intelligence*. IJCAI, 1973.

[12] S. Cho, D. Lee, *AgentRoom: Concurrent Multi-Agent Coding in a CRDT-Backed Shared Workspace*. arXiv:2608.23740, 2026.

[13] S. Zhang, Y. Shi, L. Wang, *PatchBoard: Schema-Grounded State Mutation for Reliable and Auditable LLM Multi-Agent Collaboration*. arXiv:2605.29313, 2026.

[14] B. Han, S. Zhang, *Exploring Advanced LLM Multi-Agent Systems Based on Blackboard Architecture*. arXiv:2507.01701, 2025.

[15] A. Salemi, M. Parmar, P. Goyal, Y. Song, J. Yoon, H. Zamani, T. Pfister, H. Palangi, *LLM-Based Multi-Agent Blackboard System for Information Discovery in Data Science*. arXiv:2510.01285, 2025.

[16] M. Kim, *tap: A File-Based Protocol for Heterogeneous LLM Agent Collaboration*. arXiv:2606.14445, 2026.

[17] *OpenClaw*: a self-hosted multi-agent gateway (MIT). https://github.com/openclaw/openclaw, https://docs.openclaw.ai, 2026.

[18] *Agent2Agent (A2A) Protocol*. Introduced by Google, 2025; Linux Foundation project; v1.0 specification, 2026. https://a2a-protocol.org.

[19] *Model Context Protocol (MCP)*. Anthropic; Linux Foundation Agentic AI Foundation, 2024-2026.

[20] G. J. Holzmann, *The Power of Ten: Rules for Developing Safety-Critical Code*. IEEE Computer 39(6), 2006.

[21] MISRA, *MISRA C:2012 - Guidelines for the Use of the C Language in Critical Systems*, rule 21.3.

[22] B. W. Kernighan, D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[23] A. Robbins, *Linux Programming by Example: The Fundamentals*. Prentice Hall, 2004.